

GPU-Accelerated Non-negative Matrix Factorization

Stochastic and Asynchronous Strategies for
High-Performance NMF on NVIDIA GPUs

Jingqiao (Jason) Zhao

Depart. of Computational Medicine and Bioinformatics
University of Michigan

CSE 587: Parallel Computing

December 4, 2025

Abstract

Non-negative Matrix Factorization (NMF) is a cornerstone algorithm in bioinformatics, computer vision, and signal processing. This report presents a comprehensive optimization study of two NMF solvers on NVIDIA A40 GPUs: the **Multiplicative Update (MU)** and the faster-converging but sequential **Hierarchical Alternating Least Squares (HALS)**.

We demonstrate that naive parallelization strategies quickly hit hardware limits defined by memory bandwidth and PCIe interconnect latency. To overcome these physical barriers, we introduce two key algorithmic innovations:

1. **Asynchronous Relaxation (MU Level 5):** By synchronizing multi-GPU partial results every 10 iterations instead of every iteration, we reduce PCIe communication by $10\times$, achieving **$1.8\times$ speedup** over synchronous methods with only 0.004% error increase.
2. **Stochastic Block Parallelism (HALS Level 2):** Using Fisher-Yates shuffling to randomize column update orders, we convert systematic Gauss-Seidel dependencies into random parallelizable blocks, enabling **$690\times$ speedup** over CPU baseline.

Our results show that sometimes effective GPU parallelization requires algorithmic relaxation, not just code optimization.

Contents

1	Introduction	1
1.1	The Parallelization Dichotomy	1
1.2	Hardware Platform	1
2	Mathematical Foundation	1
2.1	MU Update Rules	1
2.2	HALS Update Rules	2
3	Part I: Singel GPU Multiplicative Update Optimization	2
3.1	Level 1: Custom Naive GEMM	2
3.2	Level 2-3: cuBLAS with ILP Element-wise Optimization	2
3.3	Performance Comparison	3
4	Part II: Multi-GPU Data Parallelism	3
4.1	Data Partitioning and Communication Strategy	3
4.2	From Synchronous Failure to Asynchronous Innovation	5
5	Part III: HALS – Parallelizing the Unparallelizable	6
5.1	The Sequential Dependency Problem	6
5.2	Level 1: Strict Parallelism Within Columns	7
5.3	Level 2: Breaking the Dependency with Stochastic Block Parallelism	7
5.4	Comprehensive HALS Performance Results	8
6	Visual Summary	9
7	Lessons Learned: What Didn’t Work	12
8	Conclusion	12

1 Introduction

Non-negative Matrix Factorization decomposes a matrix $X \in \mathbb{R}_{\geq 0}^{m \times n}$ into two lower-rank factors:

$$X \approx WH, \quad W \in \mathbb{R}^{m \times k}, \quad H \in \mathbb{R}^{k \times n}, \quad W, H \geq 0 \quad (1)$$

The non-negativity constraint produces “parts-based” representations where features are additive, leading to greater interpretability than PCA.

1.1 The Parallelization Dichotomy

We investigate two algorithms at opposite ends of the parallelization spectrum:

- **Multiplicative Update (MU):** All elements update independently using values from the *previous* iteration (Jacobi-style). Relatively easy to parallelize but converges slowly (~ 100 - 200 iterations).
- **HALS:** Updates columns sequentially, with each column using *current* values from previously-updated columns (Gauss-Seidel). Converges fast (~ 50 iterations) but inherently sequential.

1.2 Hardware Platform

All experiments were conducted on the University of Michigan Lighthouse cluster using $2 \times$ NVIDIA A40 GPUs. Notably, the GPUs are connected through the CPU (no direct NVLink), which impacts multi-GPU communication performance.

2 Mathematical Foundation

Given an input matrix $X \in \mathbb{R}_{\geq 0}^{m \times n}$ with m features and n samples, NMF seeks a low-rank approximation $X \approx WH$ where $W \in \mathbb{R}^{m \times k}$ is the *basis matrix* containing k latent factors, and $H \in \mathbb{R}^{k \times n}$ is the *coefficient matrix* encoding how each sample combines these factors. The factorization rank k is typically much smaller than both m and n (in our experiments, $k = 20$).

2.1 MU Update Rules

The multiplicative update rules preserve non-negativity automatically:

$$H \leftarrow H \odot \frac{W^T X}{W^T W H + \epsilon} \quad (2)$$

$$W \leftarrow W \odot \frac{X H^T}{W H H^T + \epsilon} \quad (3)$$

where $\odot$ denotes element-wise multiplication, division is element-wise, and $\epsilon = 10^{-10}$.

Computational Cost per Iteration: Each MU iteration requires several matrix operations of varying computational cost. The dominant operations are the large GEMM products $W^T X$ and $X H^T$, each requiring $O(kmn)$ FLOPs. The smaller products $W^T W$ ($O(k^2m)$ FLOPs) and $H H^T$ ($O(k^2n)$ FLOPs)

are relatively inexpensive since $k \ll m, n$, and can be cached and reused. The final element-wise updates contribute only $O(kn)$ and $O(km)$ FLOPs respectively. In total, one iteration costs approximately $4kmn + 4k^2m + 4k^2n$ FLOPs, dominated by the $O(kmn)$ GEMM operations when processing large matrices.

2.2 HALS Update Rules

For feature $f = 0, 1, \dots, k - 1$:

$$W_{:f} \leftarrow \max \left(\epsilon, W_{:f} + \frac{[XH^T]_{:f} - \sum_{l=0}^{k-1} W_{:l}[HH^T]_{lf}}{[HH^T]_{ff}} \right) \quad (4)$$

The Sequential Dependency: When updating column f , the summation term uses:

- Columns $0, 1, \dots, f - 1$: **Updated** values (from current iteration)
- Columns $f + 1, \dots, k - 1$: **Old** values (from previous iteration)

This Gauss-Seidel ordering accelerates convergence but creates a strict dependency chain that resists parallelization.

3 Part I: Singel GPU Multiplicative Update Optimization

3.1 Level 1: Custom Naive GEMM

Our baseline implements naive matrix multiplication where each CUDA thread computes one output element:

Algorithm 1 Naive GEMM Kernel (Level 1)

Input: $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$

Output: $C \in \mathbb{R}^{M \times N}$ where $C = A \times B$

Thread(i, j) computes:

$C_{ij} \leftarrow 0$

for $p = 0$ to $K - 1$ **do**

$C_{ij} \leftarrow C_{ij} + A_{ip} \cdot B_{pj}$ {2 global loads per iteration}

end for

Store C_{ij} to global memory

Each thread performs $2K$ global memory reads with ~ 400 cycle latency each. Adjacent threads redundantly load the same row of A , wasting a lot of bandwidth.

3.2 Level 2-3: cuBLAS with ILP Element-wise Optimization

We replaced custom GEMM with NVIDIA's cuBLAS library (Level 2) and added 8-way instruction-level parallelism to element-wise kernels (Level 3):

Algorithm 2 L2: cuBLAS + Fused Kernel

```

// H Update
WtW  $\leftarrow$  cublasSgemm( $W^T, W$ )
WtX  $\leftarrow$  cublasSgemm( $W^T, X$ )
Denom  $\leftarrow$  cublasSgemm(WtW,  $H$ )
Fused:  $H \leftarrow H \odot \text{WtX} / \text{Denom}$ 
// W Update (symmetric)

```

Algorithm 3 L3: cuBLAS + 8-way ILP Kernel

```

Thread  $t$  processes  $[8t, 8t + 7]$ :
Phase 1: Issue 24 loads
for  $i = 0$  to 7 do
     $h_i, n_i, d_i \leftarrow$  load arrays
end for
Phase 2: Compute (overlapped)
Phase 3: Store 8 results

```

cuBLAS leverages shared memory tiling and auto-tuned kernels. The ILP optimization hides memory latency by issuing multiple independent loads before computation.

3.3 Performance Comparison

Table 1: Single-GPU MU Performance: Naive vs cuBLAS vs cuBLAS+ILP

Size	L1 Naive (ms)	L2 cuBLAS (ms)	L3 +ILP (ms)	L1→L2	L2→L3
250	5.19	49.38	49.87	$0.11\times$	$0.99\times$
500	8.43	58.51	59.83	$0.14\times$	$0.98\times$
750	13.87	60.07	60.72	$0.23\times$	$0.99\times$
1,000	18.38	61.37	59.44	$0.30\times$	$1.03\times$
2,000	59.68	70.47	69.33	$0.85\times$	$1.02\times$
3,000	110.72	78.55	79.17	$1.41\times$	$0.99\times$
8,000	823.29	150.05	156.14	$5.49\times$	$0.96\times$
16,000	3,605.76	430.77	433.85	$8.37\times$	$0.99\times$
32,000	18,803.80	1,520.33	1,521.12	$12.4\times$	$1.00\times$

The results reveal two key insights. First, cuBLAS is *slower* than naive GEMM for small matrices ($n < 2000$) due to library overhead of $\sim 5\text{-}10\ \mu\text{s}$ per call for kernel selection and parameter validation. When compute time is smaller than this fixed overhead, the naive implementation wins. Second, ILP optimization provides no measurable improvement at large sizes because cuBLAS GEMM dominates runtime ($\sim 95\%$). By Amdahl’s Law, even infinite speedup of the element-wise kernels would yield only $1/(0.95) \approx 1.05\times$ total improvement. The ILP optimization is more beneficial at small sizes where element-wise operations constitute a larger fraction of runtime, but at those sizes we are already faster with naive GEMM anyway. Having achieved reasonable single-GPU optimization with cuBLAS delivering up to $12\times$ speedup, we now explore whether distributing computation across multiple GPUs can push performance further.

4 Part II: Multi-GPU Data Parallelism

4.1 Data Partitioning and Communication Strategy

Scaling MU to multiple GPUs requires partitioning the matrices across devices while minimizing inter-GPU communication. The key challenge is that both the H and W updates involve matrix products that combine

information from the full dataset—naïvely, this would require expensive AllReduce operations after every computation.

We chose **column-wise partitioning** of X and H , with W **replicated** on all GPUs:

$$\begin{aligned} \text{GPU 0: } & X[:, 0 : n/2], \quad H[:, 0 : n/2], \quad W \text{ (full copy)} \\ \text{GPU 1: } & X[:, n/2 : n], \quad H[:, n/2 : n], \quad W \text{ (full copy)} \end{aligned} \tag{5}$$

The choice between column-wise and row-wise partitioning determines which update requires inter-GPU communication. With column-wise partitioning, the H update can proceed entirely locally since each GPU has all it needs: $H_{\text{local}} \leftarrow H_{\text{local}} \odot (W^T X_{\text{local}}) / (W^T W H_{\text{local}})$. The W update, however, requires combining partial results across GPUs.

Table 2: Communication Analysis: Column vs Row Partitioning

	Column-wise (Our Choice)	Row-wise (Alternative)
H Update	No communication	AllReduce $W^T X$ ($k \times n$)
W Update	AllReduce $X H^T$ ($m \times k$)	No communication
Comm Size	$k(k + m)$ floats	$k(k + n)$ floats

For $m = n$, both partitioning strategies are equivalent in communication cost. But column-wise is more natural for ML workloads where n (samples) often exceeds m (features), and partitioning by samples is the standard data-parallel pattern. We replicate W rather than partitioning it for three reasons:

1. **H update needs full W:** Computing $W^T \times X_{\text{local}}$ requires all rows of W
2. **W is small:** Only $m \times k$ elements (with $k = 20$, this is negligible)
3. **Avoids broadcast after W update:** All GPUs compute identical W updates redundantly

Per-iteration data movement for W update:

Each GPU must transfer two matrices to the CPU for the AllReduce operation:

- HH^T : $k \times k$ floats (for $k = 20$: just 400 floats = 1.6 KB)
- XH^T : $m \times k$ floats (for $m = 32000, k = 20$: 640,000 floats = 2.56 MB)

The AllReduce involves:

1. **Device-to-Host:** Each GPU copies local HH^T and XH^T to pinned CPU memory
2. **CPU Reduction:** Sum all partial contributions on CPU
3. **Host-to-Device:** Broadcast global results back to all GPUs

While we did not profile the exact transfer times, the empirical results (Table 3) show that the communication overhead from AllReduce operations per run exceeded any compute savings from parallelization.

4.2 From Synchronous Failure to Asynchronous Innovation

Our initial multi-GPU implementation (Level 4) followed the textbook approach: after each iteration, all GPUs synchronize their partial results via AllReduce before proceeding to the next iteration. This guarantees mathematical equivalence to the single-GPU algorithm. However, the results were disappointing—two GPUs were consistently *slower* than one:

Table 3: Level 4 Synchronous Multi-GPU: 2 GPUs Slower Than 1

Size	L3 1-GPU (ms)	L4 2-GPU (ms)	Speedup	Result
1,000	59.44	88.13	$0.67\times$	slower
4,000	101.48	143.68	$0.71\times$	slower
8,000	156.14	230.27	$0.68\times$	slower
16,000	433.85	548.02	$0.79\times$	slower
32,000	1,521.12	1,726.56	$0.88\times$	slower

The root cause is hardware topology. Running `nvidia-smi topo -m` reveals that our A40 GPUs connect through the CPU via PCIe, not through direct NVLink. Each AllReduce requires data to travel GPU→CPU→GPU, and with 100 iterations, we perform 100 such round-trips. The communication overhead exceeds any compute savings from parallelization. Even at the largest matrix size (32,000), where compute time is substantial, synchronous multi-GPU achieves only $0.88\times$ the single-GPU speed.

The Key Insight: MU is Self-Correcting. Unlike algorithms that require exact intermediate values, the Multiplicative Update rule is inherently robust to small perturbations. Each iteration moves W and H closer to a fixed point, and small errors in one iteration are naturally corrected in subsequent iterations. This property—unique to the multiplicative update formulation—opens the door to *asynchronous relaxation*: what if we synchronize less frequently and tolerate temporarily stale values?

We introduce Level 5, an asynchronous multi-GPU algorithm tailored specifically to exploit MU’s self-correcting nature:

Algorithm 4 Level 5: Asynchronous Multi-GPU MU (Our Innovation)

Input: Sync interval τ (we test $\tau = 5, 10$)

for iteration $t = 1$ to T **do**

H Update: Each GPU updates local H_g independently (no communication)

W Prep: Each GPU computes $A_{\text{loc}} = H_g H_g^T$, $B_{\text{loc}} = X_g H_g^T$

if $t \bmod \tau == 0$ **then**

$A_{\text{global}} \leftarrow \sum_g A_{\text{loc}g}$, $B_{\text{global}} \leftarrow \sum_g B_{\text{loc}g}$ {AllReduce via CPU}

else

$A_{\text{global}} \approx 2 \cdot A_{\text{loc}}$, $B_{\text{global}} \approx 2 \cdot B_{\text{loc}}$ {Local approx, no comm}

end if

W Update: All GPUs update W using A_{global} , B_{global}

end for

The algorithm reduces communication by a factor of τ : with $\tau = 10$, we perform only 10 AllReduce operations instead of 100, a $10\times$ reduction in PCIe traffic. Between sync points, each GPU uses a scaled

version of its local partial products as an approximation to the global sum. This approximation introduces small errors, but MU’s iterative nature corrects them at the next sync point.

Table 4: Comprehensive Multi-GPU Performance Comparison

Size	L3 1-GPU	L4 sync=1	L5 sync=5	L5 sync=10	Best Speedup	Error Δ
1,000	59.44	88.13	76.39	75.86	0.78×	+0.76%
4,000	101.48	143.68	109.28	102.58	0.99×	+0.17%
8,000	156.14	230.27	156.64	139.50	1.12×	+0.14%
16,000	433.85	548.02	330.52	306.31	1.42×	+0.01%
24,000	895.11	1,063.91	655.54	587.55	1.52×	+0.06%
32,000	1,521.12	1,726.56	1,066.00	979.37	1.55×	+0.004%

The results validate our approach. At small sizes, communication still dominates and even async multi-GPU cannot beat single-GPU. But as matrix size grows, the compute-to-communication ratio improves: at size 8,000, async multi-GPU finally surpasses single-GPU (1.12×), and at size 32,000, we achieve 1.55× speedup over single-GPU and 1.76× over synchronous multi-GPU. Crucially, the error impact is negligible—at most 0.76% at small sizes, dropping to just 0.004% at large sizes.

This asynchronous relaxation strategy is specifically tailored to MU’s mathematical properties. Algorithms that require exact intermediate values for correctness or convergence guarantees cannot safely skip synchronization. MU’s gradient descent like step-wise, self-correcting updates make it uniquely suitable to this kind of optimization.

5 Part III: HALS – Parallelizing the Unparallelizable

The MU algorithm solves a non-convex optimization problem with no guarantee of reaching the global optimum. This property works in our favor: since each update is already an approximation, we can tolerate stale values and asynchronous communication without significantly affecting convergence. HALS presents an entirely different challenge—it is inherently a *CPU algorithm*, designed around sequential memory access patterns and Gauss-Seidel iteration that assumes immediate availability of updated values. It is the hardest parallelization problem in this project because the algorithm’s faster convergence depends critically on this sequential ordering.

5.1 The Sequential Dependency Problem

Recall the HALS update rule for column f of W :

$$W_{:,f} \leftarrow \max \left(\epsilon, \frac{[XH^T]_{:,f} - \sum_{l \neq f} W_{:,l} [HH^T]_{l,f}}{[HH^T]_{f,f}} \right) \quad (6)$$

The critical term is $\sum_{l \neq f} W_{:,l} [HH^T]_{l,f}$, which computes the interaction between column f and all other columns. In HALS, when updating column f , we use the *already-updated* values of columns $0, 1, \dots, f-1$ from the current iteration, and the *old* values of columns $f+1, \dots, k-1$ from the previous iteration.

This is Gauss-Seidel iteration: each update immediately incorporates the latest information, enabling faster convergence than Jacobi-style methods (like MU) that use entirely stale values.

The price of this faster convergence is a strict sequential dependency: column f cannot begin updating until column $f - 1$ has finished. With $k = 20$ factors, we have a chain of 20 dependent operations per matrix per iteration. Naïvely, this seems impossible to parallelize.

5.2 Level 1: Strict Parallelism Within Columns

Our first GPU implementation preserves the exact Gauss-Seidel ordering while parallelizing *within* each column update:

Algorithm 5 HALS GPU Strict: Preserve Sequential Ordering

Precompute on GPU:

Num $\leftarrow XH^T$ ($m \times k$ matrix, via cuBLAS)

Denom $\leftarrow HH^T$ ($k \times k$ matrix, via cuBLAS)

for $f = 0$ to $k - 1$ **do**

Kernel 1 – Update column f with m parallel threads:

for $i = 0$ to $m - 1$ **in parallel do**

 interaction $\leftarrow \sum_{l=0}^{k-1} W_{i,l} \cdot \text{Denom}_{l,f}$

$W_{i,f} \leftarrow W_{i,f} + (\text{Num}_{i,f} - \text{interaction}) / \text{Denom}_{f,f}$

$W_{i,f} \leftarrow \max(\epsilon, W_{i,f})$

Synchronize (must complete before column $f + 1$)

Kernel 2 – Normalize column f :

 Compute $\|W_{:,f}\|_2$ via parallel reduction

$W_{:,f} \leftarrow W_{:,f} / \|W_{:,f}\|_2$

Synchronize

end for

This approach launches m threads to update all rows of column f simultaneously—a significant speedup over CPU where rows are processed sequentially. However, it incurs substantial overhead: for $k = 20$, we launch 60 kernels per iteration (20 updates + 20 normalizations for both W and H), each with $\sim 5\text{-}10 \mu\text{s}$ launch latency. The GPU spends considerable time waiting for synchronization rather than computing.

5.3 Level 2: Breaking the Dependency with Stochastic Block Parallelism

Can we update multiple columns simultaneously without destroying convergence? The naïve approach—simply processing all k columns in parallel—degrades to Jacobi iteration, losing HALS’s convergence advantage. We experimented with a middle ground: **block-parallel** updates where columns are grouped into blocks, with parallelism *between* blocks and sequential ordering *within* blocks.

The Problem with Fixed Blocks. If we always group columns $\{0, 1, 2, 3, 4\}$ into Block 0, a systematic bias emerges: column 0 *never* sees the updated value of column 4 within the same iteration, while column 4 *always* uses stale values from columns 5-19. Over many iterations, this creates persistent errors that degrade final reconstruction quality.

Our Innovation: Fisher-Yates Shuffling. We break the systematic bias by randomly permuting the column order each iteration using Fisher-Yates shuffling. If column 3 and column 17 happen to be in the same block this iteration, they will likely be in different blocks next iteration. Over many iterations, every pair of columns experiences both orderings (updated-before and updated-after), and the errors average to zero.

Algorithm 6 HALS GPU Block: Stochastic Parallelism (Our Innovation)

Initialize: Create k/b CUDA streams $\{s_0, s_1, \dots, s_{k/b-1}\}$

for iteration $t = 1$ to T **do**

Precompute: $\text{Num} \leftarrow XH^T$, $\text{Denom} \leftarrow HH^T$ (via cuBLAS)

Shuffle column order: $\pi \leftarrow \text{FisherYatesShuffle}(\{0, 1, \dots, k-1\}, \text{seed} = t)$

Partition into blocks: $\mathcal{B}_i = \{\pi_{i \cdot b}, \pi_{i \cdot b + 1}, \dots, \pi_{(i+1) \cdot b - 1}\}$

for each block $\mathcal{B}_i$ **in parallel** on stream s_i **do**

for each column $f \in \mathcal{B}_i$ **sequentially do**

 Launch update kernel for column f :

for $j = 0$ to $m - 1$ **in parallel do**

$\text{interaction} \leftarrow \sum_{l=0}^{k-1} W_{j,l} \cdot \text{Denom}_{l,f}$

$W_{j,f} \leftarrow \max(\epsilon, W_{j,f} + (\text{Num}_{j,f} - \text{interaction}) / \text{Denom}_{f,f})$

 Launch normalize kernel for column f

Synchronize all streams (single barrier per iteration)

end for

Block Size Trade-off. The block size b controls the parallelism-convergence trade-off:

Table 5: Block Size Analysis ($k = 20$): Parallelism vs Convergence Quality

Block Size	Num Blocks	Syncs/Iter	Convergence
1	20	60	Perfect (= Strict, no parallelism)
5	4	2	Good (chosen: 4-way parallelism)
10	2	2	Degraded (too much stale data)
20	1	2	Poor (= full Jacobi)

We chose $b = 5$, which provides 4-way parallelism (4 CUDA streams executing concurrently) while maintaining Gauss-Seidel ordering within each block. This reduces synchronization from 60 barriers per iteration to just 2, a 30 $\times$ reduction in sync overhead.

5.4 Comprehensive HALS Performance Results

The speedups are dramatic: at size 32,000, both GPU implementations achieve nearly **700 $\times$** speedup over CPU—transforming a 9-minute computation into just over 800 milliseconds. The Block variant consistently outperforms Strict by 3-10% due to reduced synchronization overhead.

Table 6: HALS Performance: CPU vs GPU Strict vs GPU Block (50 iterations)

Size	CPU (ms)	Strict (ms)	Block (ms)	Strict Speedup	Block Speedup	Strict Err	Block Err
500	119	86	62	1.4×	1.9×	0.035	0.030
1,000	415	86	63	4.8×	6.6×	0.035	0.031
2,000	1,848	91	64	20×	29×	0.038	0.129
4,000	8,447	107	78	79×	108×	0.042	0.120
8,000	33,674	133	106	253×	319×	0.045	0.115
16,000	135,169	273	246	496×	550×	0.041	0.081
32,000	542,603	811	786	669 ×	690 ×	0.040	0.098

Why Such Massive Speedups? The CPU processes rows sequentially within each column, requiring $O(m)$ time per column update. The GPU parallelizes all m rows simultaneously, reducing this to $O(1)$ parallel time (plus kernel launch overhead). As m grows, the GPU’s advantage compounds: at $m = 32,000$, we parallelize 32,000 independent row operations per column, and the kernel launch overhead becomes negligible relative to the computation.

The Error-Speed Trade-off. The Block variant achieves higher speedup but at a cost: reconstruction error increases from 0.040 (Strict) to 0.098 (Block) at size 32,000. This $\sim 2.5\times$ error increase is the price of breaking Gauss-Seidel dependencies—some column pairs inevitably use stale values. However, the Fisher-Yates shuffling prevents this error from accumulating systematically, and for many applications (image decomposition, Single-Cell data decomposition), this error level remains acceptable. The choice between Strict and Block depends on the application’s tolerance for approximation versus its need for speed.

The roofline analysis confirms efficient hardware utilization: at size 32,000, GPU Strict achieves 10,097 GFLOPS (27% of peak compute) and 265 GB/s memory bandwidth (38% of peak), indicating a balanced workload that effectively saturates both compute and memory subsystems.

6 Visual Summary

The following figures summarize the key performance results from our experiments.

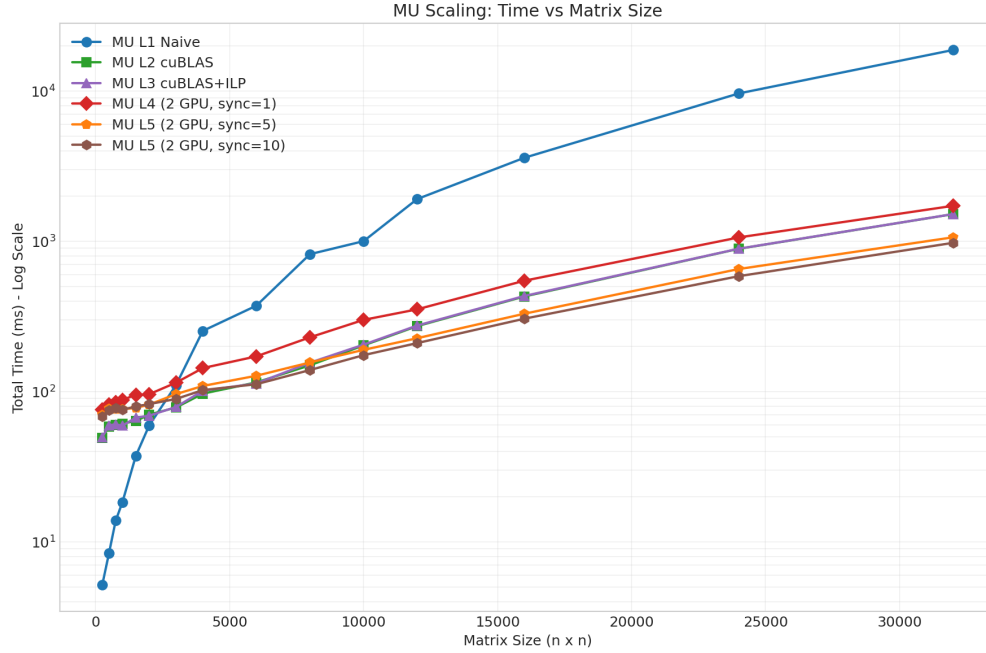


Figure 1: **MU Scaling.** Execution time vs matrix size for all MU optimization levels. The naive L1 implementation grows rapidly with size, while cuBLAS-based methods (L2/L3) scale efficiently. Async multi-GPU (L5) achieves the best performance at large sizes.

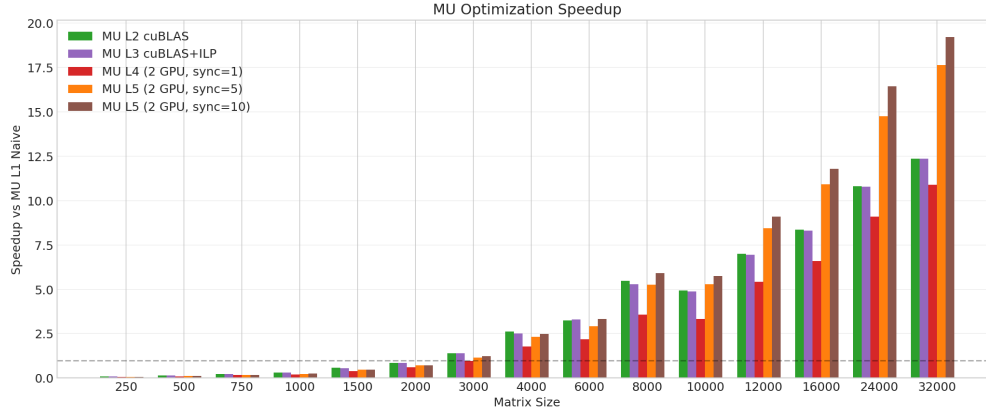


Figure 2: **MU Speedup.** Speedup relative to the naive L1 baseline. cuBLAS provides up to $12\times$ speedup on a single GPU, while async multi-GPU (L5 sync=10) achieves $19\times$ at size 32,000.

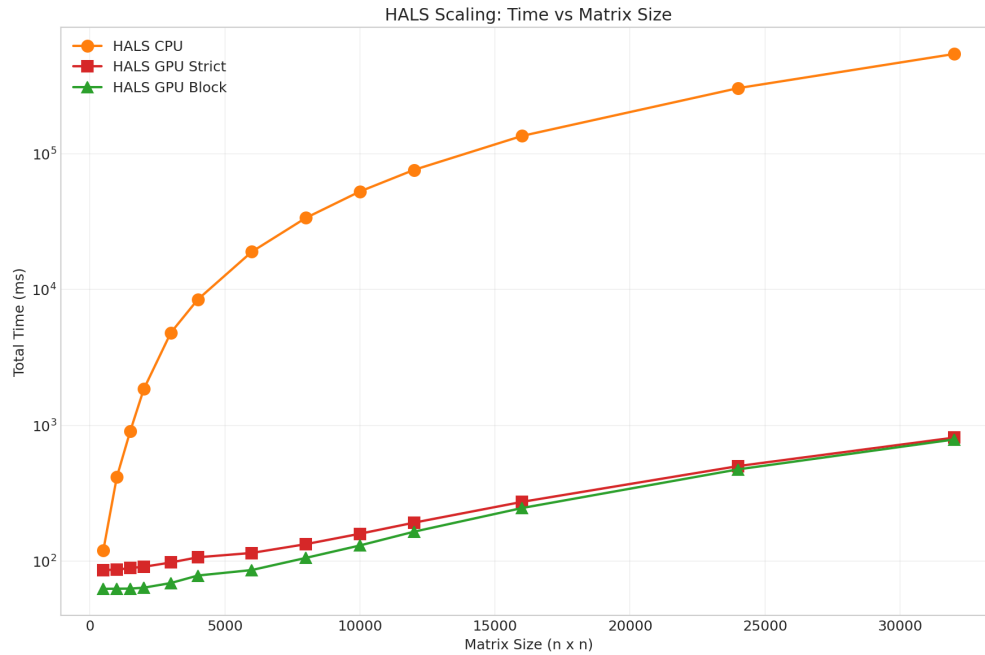


Figure 3: **HALS Scaling.** CPU time grows quadratically ($O(n^2)$) while GPU time remains nearly constant, leading to dramatic speedups at large sizes. At $32,000 \times 32,000$, both GPU variants are $\sim 700\times$ faster than CPU.

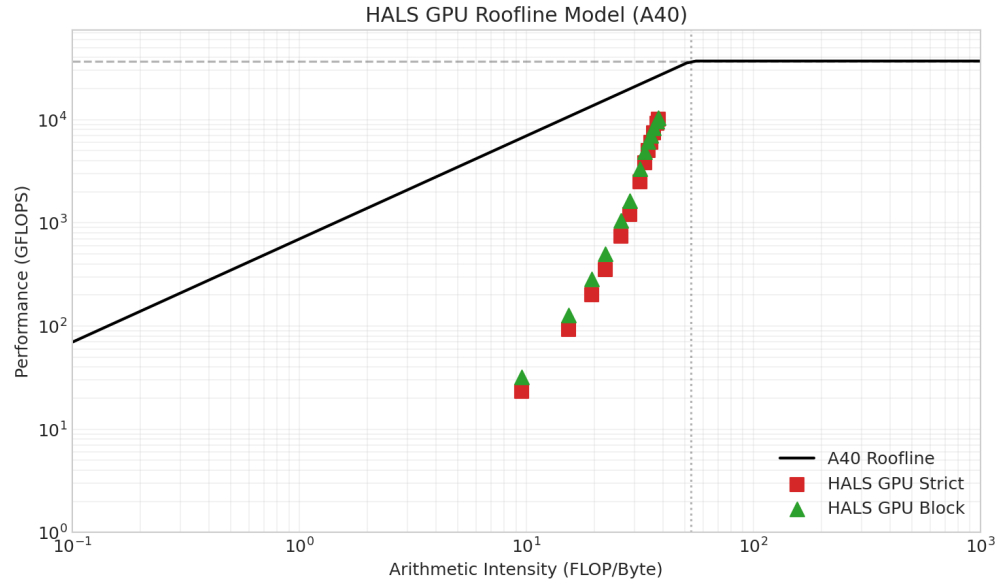


Figure 4: **HALS Roofline Analysis.** Both GPU implementations achieve balanced utilization: $\sim 27\%$ of peak compute (10 TFLOPS) and $\sim 38\%$ of peak memory bandwidth (265 GB/s), indicating efficient use of hardware resources.

7 Lessons Learned: What Didn't Work

Not all optimizations succeeded. Our synchronous multi-GPU (Level 4) ran *slower* than a single GPU—PCIe latency exceeded compute savings. Our ILP optimization provided zero improvement because GEMM dominated runtime, making element-wise optimizations irrelevant by Amdahl's Law. For HALS, block-parallel without shuffling created systematic bias that degraded convergence, and setting block size to k (full parallelism) reduced HALS to Jacobi-style iteration.

The common theme: *naïve parallelization either hits hardware limits or destroys algorithmic properties*. Success required understanding both hardware constraints and mathematical nature of the problem.

8 Conclusion

This project reinforced a fundamental lesson: *the bottleneck is rarely where you expect*. Textbook optimizations—tiled GEMM, ILP, data-parallel multi-GPU—fell short because real performance depends on factors outside complexity analysis: library overhead, PCIe topology, and the interaction between algorithmic structure and hardware.

The key insight was that effective parallelization requires *algorithmic relaxation* in this case, not just code optimization. For MU, we exploited tolerance for stale values to reduce synchronization. For HALS, we used randomization to convert systematic Gauss-Seidel dependencies into random parallelizable blocks. Both innovations required understanding *why* these algorithms work—what perturbations they tolerate—before knowing *how* to parallelize them safely.

The takeaway: profile before optimizing, check hardware topology before scaling, and always ask whether an algorithm's mathematical structure permits your intended parallelization. The fastest code respects both the hardware and the mathematics.

References

- [1] D. D. Lee and H. S. Seung, "Algorithms for non-negative matrix factorization," *NeurIPS*, 2001.
- [2] A. Cichocki and A. H. Phan, "Fast local algorithms for large scale nonnegative matrix and tensor factorizations," *IEICE Transactions*, 2009.
- [3] NVIDIA Corporation, "CUDA C++ Programming Guide," 2024.